

Lab 4

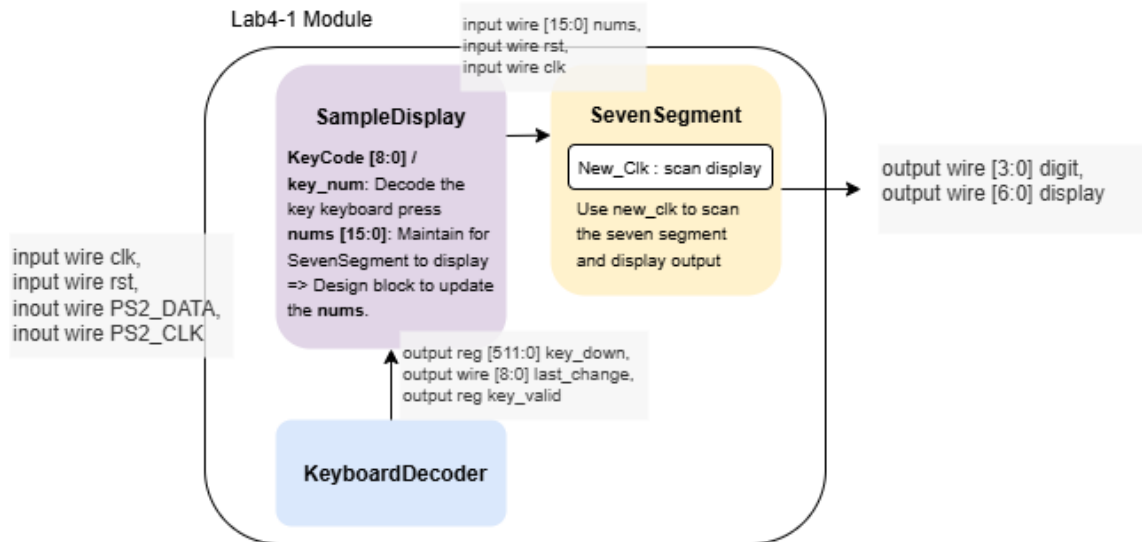
學號: 111062118

姓名: 江佩霖

A. Lab Implementation

(A) Lab4-1

i. Block Diagram



ii. Code Section

1. Sample Display

```

always @ (posedge clk, posedge rst) begin
    if (rst) begin
        nums <= 16'b1111_1111_1111_1111;
        locked <= 1'b0;
        locked_key <= 9'b0;
        sum <= 5'b000000;
    end else if (been_ready) begin
        if (!locked && key_down[last_change] == 1'b1 && key_num != 5'b11111) begin
            sum_next = (sum + key_num >= 5'b01010) ? (sum + key_num - 5'b01010) : (sum +
key_num);
            sum <= sum_next;
            nums <= {sum_next[3:0], nums[15:4]};
            locked <= 1'b1;
            locked_key <= last_change;
        end
        else if (locked && key_down[locked_key] == 1'b0) begin
            locked <= 1'b0;
        end
    end
end
assign nums_next = nums;

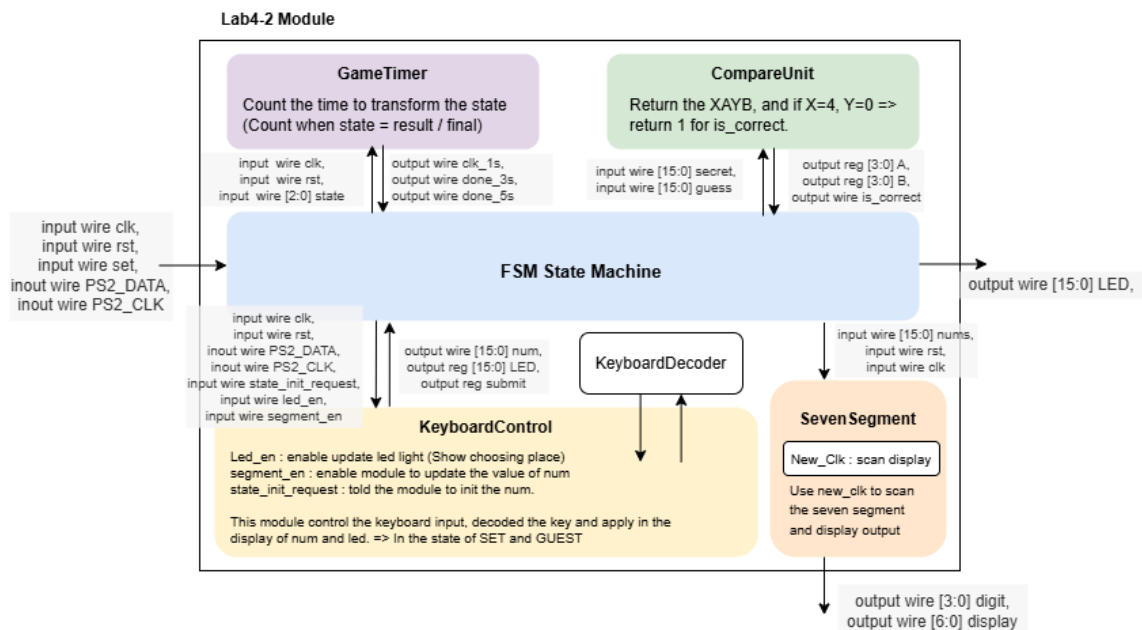
```

(1) Locked / locked_key : use to match the requirement of when a key is held down, no other key should activate until it is released.

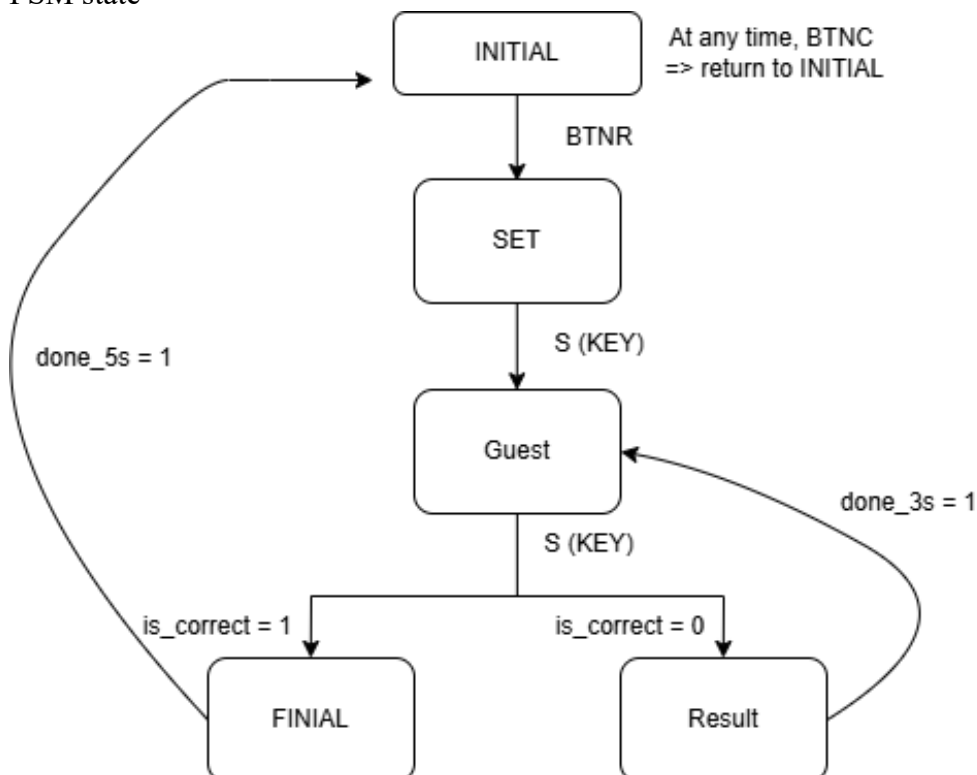
- (2) Nums : maintain the value for seven segment module to scan and decode to display the number of output. And the update of the nums will equals to the Fibonacci-Style Sequence.

(B) Lab4-2

i. Block Diagram



ii. FSM state



iii. Code Section
(1) FSM main module

```
reg [15:0] display_mux_num; // initial for INIT and Result,Final state (XAYB)
reg display_override; // 1 -> show display_mux_num, 0 -> show num_wire
// LED override buffer
reg [15:0] led_mux_num; // for INIT(display non), and Result,Final (flash)
reg led_override; // 1 -> show led_mux_num, 0 -> show led_wire
wire [15:0] display_source_num; // final output
assign display_source_num = display_override ? display_mux_num : num_wire;
assign LED = led_override ? led_mux_num : led_wire;
```

Those are for control the led and seven segment display. If the state is INIT, SET, GUESS: the override will disable so the KeyboardControl can update both output depends on the user input. While the opposite, FSM module can control the output by setting override to 1.

```
always @(*) begin
    next_state = state;
    case(state)
        S_INIT:    if(set) next_state = S_SET;
        S_SET:     if(submit_pulse) next_state = S_GUESS;
        S_GUESS:   if(submit_pulse) next_state = is_correct ? S_FINAL : S_RESULT;
        S_RESULT:  if(done_3s) next_state = S_GUESS;
        S_FINAL:   if(done_5s) next_state = S_INIT;
    endcase
end
```

The transition for the FSM, notice that “done_3s”, “5s” is the signal from GameTimer module.

```
// -----
// FSM Sequential: Output & Behavior
// -----
always @(posedge clk or posedge rst) begin //skip
```

This part is the main logic for updating all data and control the output behavior.(Total code are too long so we do not paste here, just take some special setting below)
Here we set two special signal :

```
set_flag <= 1;
guest_flag <= 1;
```

For the return state from FINAL or RESULT, the display of led and seven segment needs to be initial, keep this two value for notice the KeyboardControl to initial the value.

(2) CompareUnit

```
reg [3:0] secret_used, guess_used;
```

We define these two to mark the place that the number we already count. So it will not has any mistake that one number is count in A (place right, value right), but count twice in B (place wrong, value right).

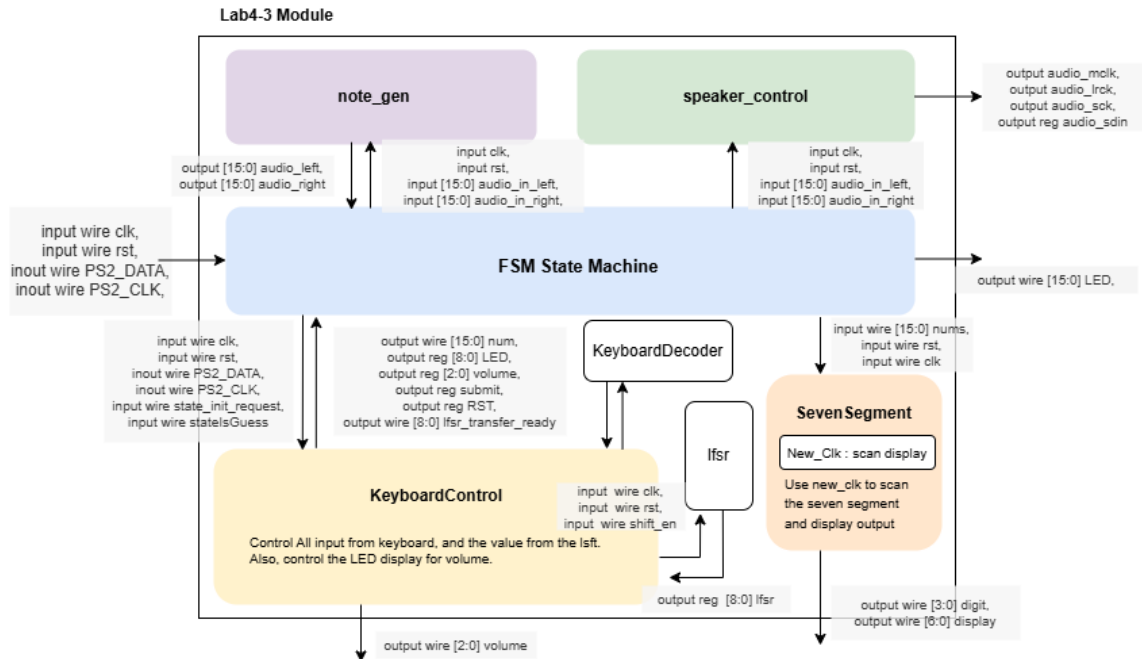
```
assign is_correct = (A==4);
```

If A is four, set the correct signal to 1.

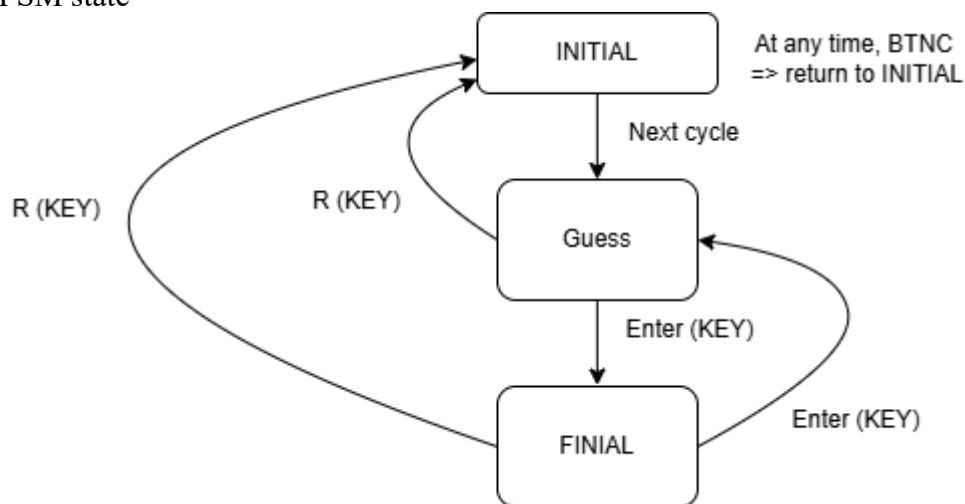
The other module's design are vary similar to Lab4-1.

(C) Lab4-3

i. Block Diagram



ii. FSM state



iii. Code Section

(1) FSM module

```

// -----
// override buffer
// -----
// display override buffer
wire [15:0] num_wire;
wire [3:0] digit_wire;
wire [6:0] display_wire;
reg [15:0] display_mux_num;
reg display_override; // 1 -> show display_mux_num, 0 -> show num_wire
// LED override buffer
wire [8:0] led_wire;

```

Same as lab4-2, design the override signal to manage the same task design in different

module.

```

wire [4:0] target_note_index = led_wire % 14;
wire [3:0] guess_note_BCD2 = num_wire[7:4];
wire [3:0] guess_note_BCD1 = num_wire[3:0];
wire [3:0] octave_offset = (guess_note_BCD1 == 4'd5) ? 4'd7 : 4'd0;
wire [4:0] guess_note_index =
    (guess_note_BCD2 >= 4'd0 && guess_note_BCD2 <= 4'd6)
    ? (guess_note_BCD2 + octave_offset)
    : 5'd31;
assign right = submit_pulse && (guess_note_index == target_note_index);
display_mux_num <= (right)? { 4'd8, 4'd12, 4'd12, 4'd5}:{ 4'd9, 4'd12, 4'd10,
4'd11 };

```

The section is to define the right signal, we use the number of the frequency of every input value, by adding the bias to check that whether the input value equals to the output of lsfr module.

```

assign note_number = led_wire % 14;
always@(*)begin
    case(note_number)
        0: freqL = 262;
        1: freqL = 294;
        2: freqL = 330;
        3: freqL = 350;
        --- (skip here)
        default:freqL = 1;
    endcase
end

```

Define of the frequency with the note number.

(2) KeyboardControl

```

// -----
// Volume Control (Sequential)
// -----
always @(posedge clk or posedge rst) begin
    if (rst) begin
        volume <= 3'd2;
    end else if (key_press_w) begin
        if (volume < 3'd3)
            volume <= volume + 1;
    end else if (key_press_s) begin
        if (volume > 3'd1)
            volume <= volume - 1;
    end
end
end

```

The control of volume.

```
// -----
// FSM Control Outputs (RST, Submit)
// -----

always @(posedge clk or posedge rst) begin
    if (rst)
        RST <= 1'b0;
    else
        RST <= key_press_r;
end
```

Assign the RST signal and return back to FSM module.

(3) note_gen

```
// Assign the amplitude of the note
// Volume is controlled here

wire signed [15:0] base_l = (b_clk == 1'b0) ? 16'hE000 : 16'h2000;
wire signed [15:0] base_r = (c_clk == 1'b0) ? 16'hE000 : 16'h2000;
reg signed [15:0] real_l, real_r;
always@(*)begin
    case(volume)
        1:begin
            real_l = base_l / 8;
            real_r = base_r / 8;
        end
        2:begin
            real_l = base_l / 2;
            real_r = base_r / 2;
        end
        3:begin
            real_l = base_l;
            real_r = base_r;
        end
        default:begin
            real_l = 0;
            real_r = 0;
        end
    endcase
end

assign audio_left = (note_div_left == 22'd1) ? 16'h0000 : real_l;
assign audio_right = (note_div_right == 22'd1) ? 16'h0000 : real_r;
```

Control volume part, generate the base frequency and divided for specific integer to generate the others.

B. Questions and Discussions

(A) In lab4_1, how did you prevent continuous detection of a positive signal when a key is pressed and held down?

When a key is pressed and held down, the keyboard controller continuously sends the same key code signal.

To avoid repeated detections of the same key press (i.e., continuous triggering), the design uses two registers — `locked` and `locked_key` — to implement a one-shot (edge-triggered) key detection mechanism.

```
if (!locked && key_down[last_change] == 1'b1)
```

`locked` is 0, meaning no key is currently registered as pressed.

`key_down[last_change] == 1'b1` indicates a new key is pressed.

At this moment:

The system executes the key's function, and sets:

```
locked <= 1'b1;
locked_key <= last_change;
```

This “locks” the key, ensuring it won't be triggered again until released.

When the key is released:

```
else if (locked && key_down[locked_key] == 1'b0)
    locked <= 1'b0;
end
```

The system detects that the same key (`locked_key`) is now released (`key_down` becomes 0), so it unlocks the detection mechanism, allowing the next press to be registered again.

(B) In lab4_3, how did you control the volume? Please explain the principle.

The volume control in Lab4_3 adjusts the amplitude of the generated audio waveform.

This is implemented through scaling the base audio signal amplitude according to the volume value.

Base wave generation :

```
assign audio_left = (note_div_left == 22'd1) ? 16'h0000 : real_l;
assign audio_right = (note_div_right == 22'd1) ? 16'h0000 : real_r;
```

These signals represent a square wave (toggle between + and - amplitude). The amplitude determines how “loud” the sound is. Ensures that when no note is played (`note_div == 1`), the output is muted. Otherwise, the scaled amplitude (`real_l`, `real_r`) is sent to the speaker output.

Amplitude scaling by volume level : (here we just show the “l” part of volume setting)

```
case(volume)
    1: real_l = base_l / 8;
    2: real_l = base_l / 2;
    3: real_l = base_l;
    default: real_l = 0;
endcase
```

The higher the volume level, the less the signal is divided → larger amplitude → louder sound.

Division is used to proportionally scale the waveform amplitude.

Besides, the keyboard control section is design in KeyboardControl module.

C. Problem Encountered

Describe the problems you encountered, solutions you developed, and the discussion. Explaining them with code segments or diagrams is recommended.

(1) Tracing and Understanding the Code

One of the major difficulties was that many parts of the project required tracing through large sections of Verilog code before modification. This process was time-consuming, especially when dealing with multiple clock domains and signal interactions.

Fortunately, the teaching assistant provided several simple template codes that clarified the basic structure of each lab module. These templates significantly reduced the setup time and allowed me to focus on debugging and feature integration more efficiently.

(2) Module Integration and Functional Overlap

Another major challenge was function coordination among multiple modules. Several features depended on signals coming from different modules—for example, keyboard input, LED control, and segment display.

Initially, I attempted to integrate all keyboard-controlled features into a single large module. However, this approach quickly became complicated and hard to maintain because the signal dependencies overlapped between music control, display, and state management.

To solve this, I introduced an override signal that served as a centralized control flag to manage module priority. When override was active, it temporarily disabled certain automatic updates, allowing manual input to take effect without signal conflicts.

D. Suggestions

沒有建議，都很棒~ 而且這次 lab 很多東西但架構滿像的(Debug 地獄...)

附上一個笑話：為什麼吃鮭魚肚子會痛？

因為鮭魚在胃食道逆流。

/* 有關 block diagram 的補充說明

有些同學對 block diagram 需要畫什麼，為什麼要畫還有疑問。

基本上 block diagram 是設計時用來表示整個系統架構的示意圖。由主要的方塊元件，還有之間的連線組成。

若構想可描繪清楚，不見得需畫出每一接線和元件。配合 block diagram 加上細部說明，就能釐清所有組成元件的詳細功能，以及彼此關連。

請注意 block diagram 與 flowchart 常用以設計「硬體或軟體」時描繪構想，因此我們讓同學在這門課程的報告裡練習，希望有助於將來用在更複雜的軟硬體設計。CS 的學生，能夠駕馭 block diagram 及 flowchart 是基本的技能！反之，很難說是可以勝任大型、實際的設計。

基本上所有設計都始於 block diagram (或 flowchart, 視設計特性而定，有時兩者都會使用)。前期的概念設計越清晰，debug 的過程會越順利。

幾項細節建議：

- 不需要把每個按鈕接上的 debounce/one-pulse 都仔細畫出裡頭在做什麼，而是把重點放在自己設計的部分。
- 概念上每一個 always block 可以是一個 block, 但如果有 FSM 可直接畫成一個方塊，再對應畫出 state diagram, 就不用把 state update 和 state transition, output 這些相關的 always block 再分別畫出來。